INTERNAL POSITIONING – EDGE AI SECURITY

Copy Fail at the Edge

CVE-2026-31431 exposure on NVIDIA Jetson / Ubuntu L4T • The case for Red Hat Device Edge

The Vulnerability in 30 Seconds

CVE-2026-31431 ("Copy Fail") is a Linux kernel local privilege escalation disclosed April 29, 2026. Any unprivileged local user can gain root in under one second. The exploit is deterministic, 732 bytes of Python, and **leaves zero forensic trace** – no disk modification, no audit log entry, no file integrity alert. Evidence is destroyed automatically on reboot.

It has existed in every mainstream Linux distribution since **2017**. CVSS 7.8 HIGH. Red Hat Bugzilla 2460538 (no errata yet).

Why This Matters at the Edge

The CVSS score says "local only." At the edge, "local" is not a meaningful barrier:

- Compromised container on the device → local access
- Vulnerable inference API or exposed SSH → local access
- Physical access to field-deployed hardware → local access
- Supply chain compromise of a model or runtime dependency → local access

Once any of these footholds exists, Copy Fail converts it to root in under one second with no evidence.

NVIDIA Jetson Is Directly Exposed

NVIDIA Jetson platforms run **L4T (Linux for Tegra)** – a full Ubuntu kernel with NVIDIA GPU drivers. The kernel is not stripped down. `CONFIG_CRYPTO_USER_API_AEAD` is enabled in the default config.

JetPack	Kernel	Base OS	Status
JetPack 6.x (Orin)	5.15 / 6.1	Ubuntu 22.04	**Vulnerable**
JetPack 5.x (Xavier / Orin)	5.10	Ubuntu 20.04	**Vulnerable**
JetPack 4.x (Nano / TX2)	4.9	Ubuntu 18.04	Below vulnerable range

The Compounding Problem at the Edge

Factor	Data Center	Edge (Jetson / Ubuntu L4T)
Patch delivery	Vendor errata → `dnf update` → hours	NVIDIA must ship new JetPack BSP → requalify → field rollout → **weeks to months**
Runtime detection	SIEM, Falco, RHACS, auditd	**None.** No security runtime on wayside/field devices
MAC enforcement	SELinux (RHEL) – custom policy can deny `AF_ALG` sockets	AppArmor (Ubuntu) – per-profile `deny network alg,` but must update every profile individually
Update mechanism	Atomic, rollback-capable	`apt upgrade` – mutable filesystem, no atomic rollback
Container isolation	seccomp + SELinux + namespace	Docker default – **no seccomp filter for `AF_ALG`**
Forensic capability	Central logging, memory forensics team	**Zero.** Exploit leaves no trace. Reboot destroys evidence.

> **THE CORE PROBLEM**
>
> When the OS kernel comes from the hardware vendor, your security posture is gated by their release schedule. NVIDIA controls the L4T kernel. Customers cannot independently patch it. Every day between disclosure and the next JetPack BSP is a day every Jetson device in the field is exploitable with a publicly available script.

What These Devices Are Doing

NVIDIA Jetson is deployed across critical infrastructure for edge AI inference. These are not lab experiments — they are production systems on OT networks:

Sector	Edge AI Workloads	Consequence of Root Compromise
Transportation	Defect detection, wayside monitoring, predictive maintenance, safety systems	Safety system manipulation, operational visibility loss, regulatory exposure
Utilities / Energy	Substation monitoring, grid edge analytics, predictive maintenance	SCADA/ICS lateral movement, grid reliability risk, NERC CIP violations
Manufacturing	Quality inspection, robotic guidance, process optimization	Production sabotage, IP exfiltration, safety bypass
Healthcare	Medical imaging, patient monitoring, pharmacy automation	HIPAA breach, patient safety risk, operational disruption

> **THE IT/OT CONVERGENCE RISK**
>
> Edge AI devices sit at the convergence point between IT and OT networks. They pull model updates and report telemetry over IP (IT side) while running inference that feeds into operational decisions (OT side). Copy Fail turns any IT-side foothold into OT-side root access with no forensic trail. This is the lateral movement path from IT into OT that every framework warns about.

Container Escape at the Edge

NVIDIA recommends containerized inference on Jetson — DeepStream, Triton Inference Server, and custom models all run in containers. This is the standard deployment pattern.

Copy Fail breaks container isolation. The Linux page cache is a host-level resource shared across all namespaces. An attacker inside a container can corrupt the page-cached copy of a setuid binary on the host, then trigger its execution.

On a Jetson running containerized inference with Docker defaults (no seccomp filter for `AF_ALG`, no SELinux, no Falco): a compromised model-serving container owns the host.

The Red Hat Alternative

Red Hat Device Edge on the same hardware provides defense in depth that the Ubuntu/L4T stack cannot match for this class of vulnerability:

Capability	Ubuntu L4T (Current)	Red Hat Device Edge
Kernel patch cadence	Gated by NVIDIA JetPack BSP releases	Red Hat errata on Red Hat's cadence, independent of hardware vendor
Update mechanism	`apt upgrade` – mutable, no rollback	`bootc` image-based – atomic, rollback-capable, testable
Mandatory Access Control	AppArmor – per-profile `deny network alg,` (no system-wide deny)	SELinux – custom policy module can deny `AF_ALG` socket creation per domain
Container runtime	Docker – default seccomp allows `AF_ALG`	Podman + custom seccomp – `AF_ALG` blocked at container boundary
Runtime security	None at the edge	Red Hat Advanced Cluster Security (RHACS) with runtime detection
Fleet management	Manual or NVIDIA Fleet Command	Red Hat Satellite + Ansible Automation Platform – patch verification at scale
Compliance	No FIPS, no STIG, no CC	FIPS 140-3 validated, STIG content, Common Criteria evaluated
Vulnerability scanning	Manual / third-party	Red Hat Lightspeed CVE advisory – automated assessment and remediation guidance

THE SELINUX STORY FOR THIS CVE

RHEL's default SELinux targeted policy allows `alg_socket` creation — the exploit works out of the box. But SELinux provides the **mechanism** to block it: a single custom policy module denying `alg_socket { create }` for `container_t`, `user_t`, and other confined domains — applied system-wide in one operation. Ubuntu's AppArmor can also deny `AF_ALG` sockets via `deny network alg,` rules, but each application profile must be updated individually. There is no system-wide equivalent to a single SELinux module. At fleet scale with hundreds of confined applications, the operational difference is significant: one policy load vs. editing every profile.

The Ask

1. **Inventory all NVIDIA Jetson deployments and their JetPack versions.** Any device on JetPack 5.x or 6.x is vulnerable.
2. **Apply the module blacklist where possible.** If the Jetson kernel has `algif_aead` as a module (not built-in), blacklist it today.
3. **Assess the containerized inference stack.** Docker on Jetson with default seccomp provides no protection against this vulnerability. Every container is a potential escape vector.
4. **Evaluate Red Hat Device Edge for edge AI workloads.** The combination of SELinux, `bootc` image mode, Podman with custom seccomp, and independent kernel patching addresses this class of vulnerability at every layer.
5. **Plan for the next one.** Copy Fail was in the kernel for nine years. The question is not whether there are more page-cache or crypto subsystem vulnerabilities — it's whether your edge devices can respond when they're found.

Talking Points

- **"Local only" is not a shield at the edge.** Any exposed service, any container vulnerability, any physical access to field hardware satisfies the local vector requirement.
- **You cannot patch what you do not control.** When the kernel comes from the hardware vendor, your security posture is their release schedule.
- **Container isolation is not a security boundary here.** The page cache is host-wide. Namespace isolation provides zero protection against page-cache corruption.
- **No forensic trail means no incident response.** If you cannot detect the exploit at runtime, you will never know it happened. How many devices have already been compromised?
- **SELinux deploys the fix system-wide; AppArmor requires per-profile changes.** Both MAC frameworks can block `AF_ALG` sockets. SELinux does it with a single custom module applied to all confined domains at once. AppArmor requires adding `deny network alg,` to every individual application profile. The default RHEL policy allows it — but one `semodule -i` command locks down every confined process on the system in seconds.

Prepared 2026-04-30 • CVE-2026-31431 • Red Hat Bugzilla 2460538 • CERT-EU 2026-005

Red Hat Enterprise Linux • Red Hat Device Edge • Red Hat OpenShift • Red Hat Advanced Cluster Security • Red Hat Satellite • Ansible Automation Platform